DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2026)

Documento de preparación para la Actividad 2

ALU y *flags*

Descripción

El presente documento tiene por objeto preparar al estudiante para la segunda actividad de laboratorio, en la que deberá describir una ALU de 8 bits. Se recomienda llegar habiendo revisado qué operaciones realiza este componente, qué son las *flags* y cómo se lee la tabla de valores de un componente entregado.

Alcance de la actividad

- **Se describe:** la arquitectura de `ALU.vhd`, es decir, la selección del resultado entre las operaciones disponibles y la generación de las *flags*.
- **Viene entregado:** el `Adder8` (con sus `FA` y `HA`), el `Reg`, el `Display_Controller` y el `Debouncer`. Estos componentes no se modifican: se importan, se declaran y se instancian leyendo su tabla de valores.
- **Se conecta:** el cableado de los componentes dentro de `Basys3.vhd`, de acuerdo con los diagramas del enunciado.

Contenidos a tener en cuenta

- **Operaciones simultáneas:** todas las operaciones se calculan en paralelo y solo una llega a la salida.
- **Fuentes de las operaciones:** el `Adder8` entregado, las compuertas lógicas y la concatenación de bits.
- **Complemento de dos:** la resta como suma del complemento del segundo operando, usando el mismo sumador.
- ***Flags*:** qué información entrega cada una y de qué depende su valor.
- **Registros y *debouncer*:** carga en el flanco de subida y uso de un botón estabilizado como reloj.

Los contenidos de `with/select`, complemento de dos, `Reg`, *debouncer* y *display* fueron trabajados en la Actividad Formativa 2, por lo que este documento solo retoma lo que la ALU agrega.

1. La ALU

Una *Arithmetic Logic Unit* es el componente que realiza las operaciones del computador. Recibe dos operandos, `a` y `b`, y un selector `sop`, y entrega un resultado junto con las *flags*.
Dentro de la ALU todas las operaciones se calculan al mismo tiempo. El circuito de la suma, el de la resta, los de `and`, `or`, `xor` y `not`, y los de los desplazamientos existen todos físicamente y operan en paralelo sobre las mismas entradas: cada uno entrega su resultado en todo momento, incluso cuando no es el que se está pidiendo. De todos esos resultados solo uno llega a la salida, y esa elección es la que realiza `sop`. Como `sop` es de 3 *bits*, se pueden distinguir ocho operaciones.

El selector: `with/select`

Esa elección se describe con una expresión **`with/select`**, que asigna a una señal un valor distinto según el valor de un selector. La misma construcción sirve para describir *muxers*, *demuxers*, *decoders* y *encoders*:

```
-- Muxer
 with mux_select select
    mux_out <= mux_in_1 when "00",
               mux_in_2 when "01",
               mux_in_3 when "10",
               mux_in_4 when "11";
-- Demuxer
 with demux_select select
    demux_out_1 <= demux_in when "00",
                   '0'      when others;
 with demux_select select
    demux_out_2 <= demux_in when "01",
                   '0'      when others;
 with demux_select select
    demux_out_3 <= demux_in when "10",
                   '0'      when others;
 with demux_select select
    demux_out_4 <= demux_in when "11",
                   '0'      when others;
-- Decoder
 with decoder_in select
    decoder_out <= "0001" when "00",
                   "0010" when "01",
                   "0100" when "10",
                   "1000" when "11";
-- Encoder
 with endecoder_in select
    encoder_out <= "00" when "0001",
                   "01" when "0010",
                   "10" when "0100",
                   "11" when "1000";
                   "00" when others;
```

- Todos los valores posibles del selector deben quedar cubiertos. El uso de `when others` permite no tener que escribir todas las opciones y evita que la síntesis infiera memoria no deseada.
- Todas las alternativas deben tener el mismo ancho que la señal asignada.
- La expresión es **concurrente**: describe un circuito permanente y no una decisión que se evalúa una sola vez. El orden en que se escriben las asignaciones dentro de la arquitectura no altera el resultado.
- Cada señal debe asignarse en un solo lugar. Dos expresiones sobre la misma señal describen dos circuitos conectados al mismo cable.

2. Operaciones

Las operaciones de la ALU se obtienen a partir de elementos que ya conocen: el `Adder8` entregado, las compuertas lógicas y la concatenación de bits. Identificar a qué familia pertenece cada operación determina cómo debe describirse.

Aritméticas: el sumador entregado

El `Adder8` es la extensión del sumador construido en la Actividad 1: ocho `FA` en cascada, cada uno construido con dos `HA`. Recibe `a(7:0)`, `b(7:0)` y un acarreo de entrada `ci`, y entrega la suma `s(7:0)` junto con el acarreo de salida `co`.
Para la resta no se necesita un componente adicional. En **complemento de dos**, restar equivale a sumar el opuesto del segundo operando, el que se obtiene invirtiendo todos sus bits y sumando 1:

$$a - b = a + (\overline{b} + 1)$$

Ese $+1$ se entrega como acarreo de entrada `ci`, por lo que ambas operaciones aritméticas utilizan la misma instancia del sumador y solo cambian sus entradas. Si el resultado no cabe en 8 bits, el bit que sobra sale por `co`. En otro caso, el acarreo de salida `co` es igual a cero.

Lógicas: bit a bit

Las operaciones `and`, `or`, `xor` y `not` trabajan sobre vectores **bit a bit**: el bit i del resultado depende únicamente de los bits i de los operandos, sin acarreo ni relación entre un bit y su vecino. Además, `not` es una operación de un solo operando: si la tabla indica `not a`, entonces `b` no participa.

Desplazamientos: concatenación

Un desplazamiento se describe con el operador de concatenación `&`, que une bits y vectores para formar uno más largo:

- `'0' & a(7 downto 1)` toma los siete bits altos de `a`, los baja una posición y rellena por arriba con un cero.
- `a(6 downto 0) & '0'` sube los siete bits bajos y rellena por abajo con un cero.

En ambos casos el resultado mantiene los 8 bits, porque se agrega un cero por un extremo mientras un bit queda fuera por el otro. Ese bit no se descarta: la tabla de valores indica dónde debe aparecer.

3. Leer la tabla de valores

La actividad no explica en palabras lo que la ALU debe hacer: entrega una **tabla de valores**, que dice qué valor toma cada salida para cada combinación de las entradas. Es el contrato que la descripción en VHDL debe cumplir. Los ejemplos que siguen usan un componente genérico, que no es la ALU pero se lee igual.

Paso 1: separar las entradas de las salidas

La línea doble parte la tabla en dos: a su izquierda, lo que el componente recibe; a su derecha, lo que debe producir.

Entradas			Salidas	
`entrada_1`	`entrada_2`	`sel`	`salida`	`flag_1`

En esta tablita las cinco columnas son exactamente los puertos de la entidad: la tabla y el `port` del archivo VHDL siempre coinciden. En la ALU las entradas son `a`, `b` y `sop`, y las salidas son `result` y las tres *flags*.

Paso 2: leer cada fila como una frase

Una fila no es una instrucción que se ejecuta, sino una afirmación que siempre debe cumplirse.

`entrada_1`	`entrada_2`	`sel`	`salida`	`flag_1`
`*`	`*`	`"10"`	`entrada_1 and entrada_2`	`entrada_1(7)`

Esta tablita se lee: *cuando* `sel` *vale* `"10"`, `salida` *debe ser* `entrada_1 and entrada_2` *y* `flag_1` *debe ser el bit 7 de* `entrada_1`. Lo que está a la izquierda de la línea doble es la condición; lo que está a la derecha, lo que el circuito debe entregar.

Paso 3: ver qué entradas participan

No todas las filas usan los dos operandos.

`sel`	`salida`
`"01"`	`not entrada_1`
`"10"`	`entrada_1 and entrada_2`

En esta tablita, la fila de `"01"` no menciona `entrada_2`, así que esa alternativa se describe usando solo `entrada_1`; la de `"10"` sí usa las dos.

Paso 4: ver de dónde sale cada valor

Las casillas de salida no son todas del mismo tipo.

Lo que dice la casilla	Qué es
`"00000000"`	una constante
`entrada_1(7)`	un bit de una entrada
`entrada_1 and entrada_2`	el resultado de un circuito que hay que describir

Clasificar así cada casilla ahorra trabajo: las dos primeras filas ya existen en el circuito y solo hay que conectarlas.

Paso 5: interpretar el símbolo *

El * aparece solo en columnas de entrada y significa *cualquier valor*.

entrada_1	entrada_2	sel	salida	flag_1
*	*	"00"	entrada_1	'0'
*	*	"01"	not entrada_1	'0'
*	*	"10"	entrada_1 and entrada_2	entrada_1(7)
*	*	"11"	"00000000"	'1'

En esta tablita las cuatro filas llevan * en los dos operandos porque cada operación queda definida para cualquier par de valores: la fila de "01" vale tanto si entrada_1 es "00000000" como si es "10110101". La única entrada que cambia de fila en fila es sel; en la ALU, sop.
Ojo con una confusión frecuente: que una entrada lleve * no significa que la operación la ignore. En la fila de "10" las dos entradas llevan * y ambas aparecen en la salida. Compare con esta otra tablita:

clock	clear	load	dataout
*	1	*	"00000000"

Aquí el * sí marca entradas que no influyen, porque con clear activo el registro se va a cero pase lo que pase con las demás. Cuál de los dos casos aplica se decide leyendo la fila completa, nunca la columna aislada. En las columnas de salida el * no aparece: una salida es un cable que siempre tiene un valor.

De la tabla a la descripción

Cada columna de salida se convierte en una expresión with/select y cada fila en una de sus alternativas.

sel	salida	flag_1
"00"	entrada_1	'0'
"01"	not entrada_1	'0'
"10"	entrada_1 and entrada_2	entrada_1(7)
"11"	"00000000"	'1'

Esta tablita es la misma de antes sin las columnas con *, y se describe así:

```
-- Una expresion por cada columna de salida.
with sel select
    salida <= entrada_1               when "00",
              not entrada_1           when "01",
              entrada_1 and entrada_2 when "10",
              "00000000"              when others;

with sel select
    flag_1 <= entrada_1(7) when "10",
              '1'          when "11",
              '0'          when others;
```

entrada_1 y entrada_2 no aparecen en ningún when porque su * indica que no son parte de la condición: la condición la impone solo sel.
Para cada fila de la tabla de la ALU quedan entonces tres preguntas: qué operandos participan, a qué familia pertenece la operación (aritmética, lógica o desplazamiento) y qué señal del circuito contiene el valor que la fila pide.

4. Flags

Una *flag* es una señal de un bit que entrega información sobre el resultado de la operación. Esta ALU entrega tres, con los nombres habituales:

- `c`, de *carry*: se relaciona con la información que no cabe en los 8 bits del resultado.
- `z`, de *zero*: se relaciona con que el resultado sea igual a cero.
- `n`, de *negative*: se relaciona con el signo del resultado y con la comparación entre los operandos.

Sin ellas, un `"00000000"` no distinguiría una suma desbordada de la resta de dos números iguales, y el bit que se pierde en un desplazamiento no quedaría registrado (figura 1).

	11111111
+	00000001
1	00000000
noveno bit	resultado de 8 bits

a	10000001
'0' & a(7 downto 1)	01000000
bit que queda fuera	a(0) = 1

Figura 1: En los dos casos hay un bit que no aparece en el resultado.

Las *flags* en la tabla de valores

Las tres aparecen como columnas adicionales de la tabla, y conviene revisarlas por columna antes que por fila: la forma de la columna indica cómo describir la *flag*. Hay dos casos posibles (figura 2).

sel	flag_1
"00"	salida = 0
"01"	salida = 0
"10"	salida = 0

No depende de la operación: se describe una sola vez a partir de la salida.

sel	flag_2
"00"	0
"01"	entrada_1(7)
"10"	senal_x

Depende de la operación: necesita su propia selección sobre `sel`.

Figura 2: Una columna con el mismo valor en todas las filas y una columna que cambia de fila en fila.

Los valores que aparecen en cada casilla son además de distinto tipo:

Tipo de valor	Cómo se ve en la tabla
Una constante	`0` o `1`, cuando la operación no entrega información
Un bit de un operando	`entrada_1(7)`
Una señal de un componente	la salida de un componente instanciado
Una propiedad de la salida	`salida = 0`

Clasificar cada casilla permite notar que la mayoría de las *flags* salen de señales que ya existen en el circuito.
Es posible que alguna columna pida comparar los operandos. Antes del laboratorio conviene hacer en papel dos restas de 8 bits en complemento de dos, una con el primer operando mayor y otra con el primero menor, y observar qué ocurre con el acarreo de salida.

5. Entorno en la placa

La ALU se prueba dentro de una calculadora montada en la placa. Dos registros almacenan los operandos, un *muxer* determina si esos registros se cargan desde los interruptores o desde el resultado de la ALU, lo que permite **acumular**, y el *display* de 7 segmentos muestra los operandos o el resultado.

El registro y el botón como reloj

Un registro es un pequeño bloque de memoria dentro del circuito que almacena datos. Recibe, guarda y envía datos coordinado por un reloj: la señal `clock` es la entrada del reloj, y el registro solo actúa en su **flanco de subida**, es decir, en el instante en que la señal pasa de nivel bajo a nivel alto.

Entradas						Salidas
`clock`	`clear`	`load`	`up`	`down`	`datain(7:0)`	`dataout(7:0)`
*	1	*	*	*	*	"00000000"
Flanco Subida	0	1	*	*	*	datain
Flanco Subida	0	0	1	*	*	dataout + 1
Flanco Subida	0	0	0	1	*	dataout - 1
*	*	*	*	*	*	dataout

Figura 3: Tabla de valores de `Reg`. Las filas se leen en orden: la primera que aplica es la que manda.

La tabla se lee de la siguiente forma: `clear` tiene prioridad y vuelve la salida a cero sin necesidad de flanco; con `clear` en `'0'` y `load` en `'1'`, en el flanco de subida se guarda `datain`; y si no hay flanco, o ninguna señal de control está activa, el registro conserva su valor. Con `load` fijo en `'1'`, la señal conectada a `clock` es la que determina cuándo se realiza la carga.
Al apretar un botón la señal rebota antes de estabilizarse en su nuevo valor, y cada rebote es un flanco de subida, por lo que el registro cargaría varias veces con una sola pulsación. Por este motivo la señal del botón debe pasar primero por el *debouncer*, que no la transfiere hasta que sea estable por un periodo de tiempo. Esa señal estabilizada es la que sirve de reloj y por convención se llama `d_btn`. En `Basys3.vhd` los `Debouncer` ya vienen instanciados.

Qué controla cada entrada de la placa

- `sw(7:0)` entrega el valor que se quiere cargar en un registro.
- `sw(15:13)` corresponde al `sop` de la ALU: tres interruptores, ocho operaciones.
- `sw(12)` determina si los registros se cargan desde los interruptores o desde el resultado de la ALU.
- `btn(2)` y `btn(3)` cargan el registro `A` y el registro `B` respectivamente.
- `btn(0)` determina qué muestra el *display*: los operandos o el resultado.
- `led(15)`, `led(14)` y `led(13)` muestran las *flags*.

Errores frecuentes

A continuación se listan los errores más frecuentes en esta actividad. Los ejemplos usan nombres genéricos (`sel`, `salida`, `valor_1`) y no corresponden a la tabla de la ALU: ilustran únicamente la forma de la descripción. Los tres primeros errores corresponden al razonamiento sobre las *flags* y los demás a la descripción en VHDL.

- Combinar con `or` todas las señales que podrían producir una *flag*. El valor que pide cada fila se selecciona según `sop` y no se combina con el de las demás operaciones.

```
-- Incorrecto: mezcla en una sola senal los valores de todas las alternativas
salida <= valor_1 or valor_2 or valor_3;
-- Correcto: cada alternativa entrega su propio valor
with sel select
    salida <= valor_1 when "00",
              valor_2 when "01",
              ...     when others;
```

- Traducir literalmente a VHDL las condiciones que aparecen en la tabla. Una condición como `a >b` indica cuándo la salida debe tomar cierto valor, y no una comparación que el circuito deba ejecutar: esa condición se obtiene con lógica combinacional, a partir de las señales que ya existen.

```
-- Incorrecto: la condicion de la fila escrita como una comparacion
salida <= '1' when entrada_1 < entrada_2 else '0';
-- Correcto: la condicion la responde una senal del circuito
with sel select
    salida <= senal_que_responde_la_condicion when "01",
              '0'                             when others;
```

- Describir una *flag* de la misma forma para todas las operaciones. Salvo que su columna tenga el mismo valor en todas las filas, la *flag* depende de `sop` tanto como el resultado.

```
-- Incorrecto: un solo valor para todas las alternativas
salida <= entrada_1(7);
-- Correcto: el valor se elige segun el selector
with sel select
    salida <= entrada_1(7) when "00",
              ...          when others;
```

- Dejar valores del selector sin cubrir, en lugar de cerrar la expresión con `when others`.

```
-- Incorrecto: quedan valores de sel sin cubrir
with sel select
    salida <= valor_1 when "00",
              valor_2 when "01";
-- Correcto: la ultima alternativa cubre el resto de los casos
with sel select
    salida <= valor_1 when "00",
              valor_2 when "01",
              ...     when others;
```

- Mezclar anchos. Un `std_logic` guarda un solo bit, que se escribe entre comillas simples, y un `std_logic_vector` guarda varios, entre comillas dobles.

```
-- bit_salida : out std_logic;  vector_salida : out std_logic_vector (7 downto 0);
-- Incorrecto
bit_salida <= "0";          vector_salida <= '0';
-- Correcto
bit_salida <= '0';          vector_salida <= "00000000";
```

- Asignar la misma señal en dos expresiones distintas, con lo que quedan dos circuitos conectados al mismo cable.

```
-- Incorrecto: salida se asigna dos veces
salida <= '0';
with sel select
    salida <= valor_1 when "00",
              ...     when others;
-- Correcto: una sola expresion define salida
with sel select
    salida <= valor_1 when "00",
              ...     when others;
```

- Leer un puerto de salida. Si un valor se necesita en más de un lugar, debe guardarse en una señal interna.

```
-- vector_salida : out std_logic_vector (7 downto 0);
-- Incorrecto: un puerto de salida no puede leerse
otra_senal    <= not vector_salida;
-- Correcto: se lee la senal interna y el puerto solo se escribe
otra_senal    <= not senal_interna;
vector_salida <= senal_interna;
```

- Usar `btn` en lugar de `d_btn` como reloj de un registro.

```
-- Incorrecto
inst_REG: Reg port map( clock => btn(2), ... );
-- Correcto
inst_REG: Reg port map( clock => d_btn(2), ... );
```

- Dejar puertos sin conectar en un `port map`.

```
-- Incorrecto: falta conectar un puerto del componente
inst_comp: Componente port map( x => senal_1, y => senal_2 );
-- Correcto: todos los puertos del componente quedan conectados
inst_comp: Componente port map( x => senal_1, y => senal_2, z => senal_3 );
```

Recordatorio de VHDL

- VHDL describe hardware concurrente: todas las expresiones de la arquitectura ocurren siempre y al mismo tiempo.
- `'0'`/`'1'` corresponden a `std_logic`; `"01"` corresponde a un `std_logic_vector`.
- Operadores frecuentes: `and`, `or`, `xor`, `not`; sobre vectores operan bit a bit.
- Concatenación: `'0' & a(7 downto 1)` desplaza a la derecha; `a(6 downto 0) & '0'` desplaza a la izquierda.
- Acceso a un bit: `vec(0)`; a un rango: `vec(7 downto 4)`.
- Reutilización de un diseño: `component` + `signal` + `port map`.
- En `Basys3.vhd`, los puertos `sw`, `btn`, `led`, `seg` y `an` no se modifican; solo cambia lo que se instancia y cómo se conecta.

Material extra: esqueleto de `ALU.vhd`

A modo de estudio previo se incluye el esqueleto del archivo `ALU.vhd`: la biblioteca, la entidad con sus entradas y salidas, y la arquitectura vacía. La entidad ya está completa y no debe modificarse, porque sus puertos son los que `Basys3.vhd` espera encontrar al instanciar el componente. Todo lo que falta va entre `begin` y `end Behavioral`, y sale de la tabla de valores que entrega la actividad.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- ALU de 8 bits.
--   a, b : operandos
--   sop  : selector de operacion (3 bits, 8 operaciones)
--   c    : flag de carry
--   z    : flag de zero
--   n    : flag de negative
entity ALU is
    Port (
        a      : in  std_logic_vector (7 downto 0);
        b      : in  std_logic_vector (7 downto 0);
        sop    : in  std_logic_vector (2 downto 0);
        c      : out std_logic;
        z      : out std_logic;
        n      : out std_logic;
        result : out std_logic_vector (7 downto 0)
    );
end ALU;

architecture Behavioral of ALU is

    -- Declaracion de los componentes que se instancian (completar)

    -- Declaracion de las senales internas (completar)

begin

    -- Descripcion de las operaciones (completar)

    -- Seleccion del resultado segun sop (completar)

    -- Descripcion de las flags c, z y n (completar)

end Behavioral;
```

Antes del laboratorio conviene revisar el esqueleto y responder, para cada línea que dice `completar`, qué debería ir ahí: qué componentes se necesitan, qué señales internas hacen falta y cuántas expresiones `with/select` tendrá la arquitectura.